USD8 Booster NFT — Independent Audit

Repository: `Usd8-fi/usd8-boosters-NFT` (github.com/Usd8-fi/usd8-boosters-NFT) **Commit reviewed**: `main` HEAD as of 2026-04-27 **Scope**: `src/USD8Booster.sol` (sole contract, 136 LOC), `test/USD8Booster.t.sol`, `script/USD8Booster.s.sol`, `metadata/1.json` **Toolchain**: Solidity 0.8.28, OpenZeppelin Contracts v5, Foundry, no `via_ir`, optimizer 200 runs

---

Executive Summary

The contract is a clean ERC-1155 booster NFT with two mint paths — admin-direct and EIP-712 signature-authorized claim — plus standard burn semantics. The implementation is tight: state changes occur before external calls, signature recovery uses canonical OpenZeppelin ECDSA (rejects high-`s` malleable signatures), nonce-tracking prevents replay, the EIP-712 domain separator handles fork chains correctly via OpenZeppelin's auto-rebuild, and all admin functions are correctly gated.

Zero critical or high findings. The contract does what it says.

The audit surfaces two **medium-severity** design concerns that are not exploits today but constitute long-term operational hazards as the booster system scales: (M-01) signature-claim authorizations have no expiry, meaning leaked sigs are claimable forever absent admin invalidation, and (M-02) the token-ID → boost-percentage mapping lives in NatSpec and metadata files rather than on-chain, decoupling on-chain truth from the off-chain logic that consumes it. Neither blocks production deployment; both are worth resolving before the first new booster tier ships.

Four **low-severity** items concern defensive coding, gas DoS surface, event indexing strategy, and repository hygiene. Three **informational** items concern style and a stray ML model config file. Recommended fixes are bounded and non-controversial.

The test suite is appropriately scoped to the contract's surface — 30 tests covering admin gating, mint/burn semantics, signature recovery edge cases, signer rotation invalidating stale sigs, nonce reuse, and tampered-payload rejection. There is no fuzzing of the signature path, which is a recommended addition (see I-04).

The contract is **production-ready as a static booster issuance mechanism**. The medium findings should be addressed before the system grows beyond the single 1% tier.

Findings

Medium

M-01 — Claim signatures have no expiry

Location: `USD8Booster.sol:38-39` (typehash), `:107-123` (claim function)

Description: The `Claim` typehash binds `(receiver, nonce, tokenId, amount)` only — no `deadline`, no `validUntil`, no `issuedAt`. A signature, once produced by the admin or signer, is valid in perpetuity until either (a) the holder uses it, (b) the admin proactively calls `invalidateNonce`, or (c) the signer is rotated (which only invalidates signatures from the old signer, not the admin).

Impact: Operationally, this means every signature ever generated is a contingent liability. If a sig leaks (logs, debug output, intercepted email, leaked database, retired employee with archive access), it can be claimed years later. The mitigation today is `invalidateNonce`, but that requires the admin to be aware of the leak and to invalidate proactively. Sigs that were never delivered to their intended recipient (e.g., sent to a wrong email and intercepted) may not be known to the admin at all.

The asset at stake is bounded — each NFT is a 1% Cover Score boost — so the financial impact of any single leak is small. But the *distribution* of risk over time is unbounded: every issuance adds permanent risk that can only be retired by per-nonce admin action.

Recommendation: Add a `deadline` field to the typehash and revert if `block.timestamp > deadline`:

```
bytes32 public constant CLAIM_TYPEHASH =
    keccak256("Claim(address receiver,uint256 nonce,uint256 tokenId,uint256 amount,uint256 deadline)");

function claim(
    address receiver, uint256 nonce, uint256 tokenId, uint256 amount,
    uint256 deadline, bytes calldata signature
) external {
    if (block.timestamp > deadline) revert SignatureExpired();
    // ... rest unchanged
}
```

This is a breaking ABI change. If any signatures have already been issued against the current typehash (mainnet or testnet), they would need to be re-issued under the new format.

M-02 — Token-ID → boost-percentage mapping is off-chain

Location: `USD8Booster.sol:27-29` (NatSpec comment), `metadata/1.json` (off-chain JSON)

Description: The contract supports unlimited token IDs (`mint(address to, uint256 tokenId, uint256 amount)` accepts any `tokenId`) but only encodes the meaning of token ID 1 in a NatSpec comment ("1 = 1% cover score booster"). The boost percentage actually applied to the Cover Score is computed off-chain by USD8's frontend or backend, presumably by reading NFT balances and multiplying by some hard-coded or off-chain-configured rate.

Impact: Two failure modes.

First, **divergence**: if a future deployment introduces token ID 2 = 5% booster (per the NatSpec's "Add future token IDs here as new booster tiers are introduced"), and the off-chain logic is updated to recognize tier-2 NFTs, but a different consumer of the contract (a third-party indexer, a frontend mirror, a DAO dashboard) is not updated, the consumers see different effective boost percentages. There is no on-chain truth to fall back to.

Second, **silent misissuance**: an admin error that mints token ID 999 (intended internally as "tier 9 = 99%") becomes immediately wallet-visible as an NFT, and the off-chain logic decides what it's worth. If the off-chain logic doesn't

recognize the ID, the user sees a boost in their wallet that doesn't apply at claim time. If it *does* recognize the ID with a higher rate, the misissue can't be reversed except by burning the NFTs (admin can't burn arbitrary holders' tokens).

Recommendation: Encode boost basis points per token ID in storage, settable by admin only when `totalSupply(tokenId) == 0`:

```
mapping(uint256 tokenId => uint16 boostBps) public boostBps;

error TokenAlreadyIssued();

function setBoostBps(uint256 tokenId, uint16 bps) external onlyAdmin {
    if (totalSupply(tokenId) > 0) revert TokenAlreadyIssued();
    boostBps[tokenId] = bps;
}
```

This makes the boost semantics on-chain authoritative. Off-chain consumers query `boostBps(tokenId)` rather than relying on a maintained off-chain table. The "set only when supply is zero" guard prevents post-hoc changes that would silently re-rate already-issued boosters.

Low

L-01 — Defensive: explicit `signer != address(0)` check in `claim`

Location: `USD8Booster.sol:117`

Description: When `signer` is uninitialized (the contract starts with `signer == address(0)` per `test_SignerStartsAsZero`), the claim path matches *only* admin-signed claims. The line `if (recovered != signer && recovered != admin) revert InvalidSignature();` would, in principle, accept a signature whose `ECDSA.recover` returned `address(0)` while `signer == address(0)`. This is not exploitable in practice because OpenZeppelin's `ECDSA.recover` reverts on invalid signatures rather than returning `address(0)` — but the defensive check would make intent explicit and would survive any future migration to a `tryRecover`-style signature recovery.

Recommendation: Either add `if (signer == address(0) && recovered != admin) revert InvalidSignature();` as an explicit early branch, or short-circuit the check by treating `signer == address(0)` as "signer disabled":

```
bool signerActive = signer != address(0);
if (recovered != admin && !(signerActive && recovered == signer)) {
    revert InvalidSignature();
}
```

L-02 — `mintBatch` has no array length cap

Location: `USD8Booster.sol:77-87`

Description: `mintBatch` loops over `recipients.length` calling `_mint` for each. With sufficiently large arrays, the call exceeds the block gas limit and reverts. Because `mintBatch` is `onlyAdmin`, the only DoS target is the admin themselves — they can submit a transaction that fails to land.

Impact: Operational nuisance, not a vulnerability. An admin running batch issuance against a fresh holder list might learn the hard way that the list is too large.

Recommendation: Add an explicit `MAX_BATCH = 256` (or similar) and revert if exceeded, with a clear error message. Optional but improves operational robustness.

L-03 — `Claimed` event indexing strategy is suboptimal

Location: `USD8Booster.sol:44`

Description: The event indexes `receiver`, `tokenId`, `amount` — but not `nonce`. In practice, off-chain consumers will want to query by `nonce` (e.g., "did this signature ever get claimed?") and rarely by `amount`. The current strategy uses one of the three indexable slots for a field that is unlikely to filter usefully.

Recommendation: Swap `amount` and `nonce`:

```
event Claimed(address indexed receiver, uint256 indexed nonce, uint256 indexed tokenId, uint256 amount);
```

L-04 — No `LICENSE` file in repository

Location: Repository root

Description: `USD8Booster.sol` declares `// SPDX-License-Identifier: MIT`, but the repository has no top-level `LICENSE` file. GitHub renders the repo as "no license" and downstream consumers may be uncertain whether the MIT declaration applies to the entire repository (tests, scripts, metadata) or only to the contract.

Recommendation: Add a top-level `LICENSE` file containing the MIT license text.

Informational

I-01 — Stray `Modelfile` committed

Location: Repository root, `Modelfile`

Description: An Ollama `Modelfile` referencing a local `gemma-4-E4B-it-OBLITERATED-Q4_K_M.gguf` weight file is committed to the repository root. The "OBLITERATED" suffix denotes a fine-tune with safety filters removed. This is presumably a developer's local AI tooling configuration and is unrelated to the contract.

Recommendation: Delete the file from the repository and add `Modelfile` to `.gitignore`. Public visibility of an "OBLITERATED" model reference may invite questions you'd rather not answer in audit conversations.

I-02 — Missing NatSpec on `_update` override

Location: `USD8Booster.sol:130-135`

Description: The `_update` override has no NatSpec. While its purpose is obvious to a Solidity reviewer (combining ERC1155 + ERC1155Supply behaviors), an explicit one-line NatSpec helps automated documentation generators.

Recommendation: Add `/// @dev Required override to combine ERC1155 and ERC1155Supply._update behaviors.`

I-03 — Consider a “batch invalidate by deadline” pattern

Location: `USD8Booster.sol:95-99`

Description: If M-01 (signature deadlines) is *not* adopted, the operational burden of `invalidateNonce` grows with the number of in-flight signatures. A batch-invalidate-by-deadline function (e.g., `invalidateNoncesIssuedBefore(uint256 cutoff)` paired with on-chain issuance tracking) could reduce that burden — but it would require the contract to know when each nonce was issued, which it does not currently track. This is mentioned only as a fallback if M-01 is rejected on ABI-stability grounds.

I-04 — No fuzz tests on the signature path

Location: `test/USD8Booster.t.sol`

Description: The test suite is well-scoped for known cases (tampered fields, nonce reuse, signer rotation, zero-address rejection) but does not include fuzz tests on the EIP-712 path. Foundry fuzzing on `(receiver, nonce, tokenId, amount, signature)` could surface edge cases in domain separator handling on forked chains, large-uint nonce collisions, or unusual ERC-1155 receiver behavior.

Recommendation: Add a fuzz test along the lines of `testFuzz_ClaimRoundtrip(address receiver, uint96 nonce, uint16 tokenId, uint96 amount)` that signs and claims, asserting correctness of state transitions.

What the contract does well

This section is included because audit reports should distinguish “didn’t find anything to flag” from “didn’t look carefully enough.” These properties were verified during the review.

- **State updates precede external calls.** `nonceUsed[nonce] = true` runs before `_mint(receiver, ...)` in `claim`. A malicious ERC-1155 receiver implementing `onERC1155Received` cannot re-enter `claim` with the same nonce. The standard checks-effects-interactions ordering is followed throughout.

- **OpenZeppelin ECDSA enforces canonical signature form.** High-`s` signature malleability is rejected at the recover step. No need for a defense in `claim` itself.
- **EIP-712 domain separator handles fork chains.** OpenZeppelin's `EIP712` rebuilds the domain separator if the cached `chainId` or contract address differs from the live values, which prevents cross-chain replay after a hard fork.
- **Admin is immutable; signer is rotatable.** This is the right split. Compromise of the operational signer is recoverable; compromise of the admin is by design unrecoverable, which forces the admin key to live in the most secure custody available.
- **`invalidateNonce` exists.** Many contracts of this shape ship without an emergency-invalidation surface and discover the gap during incident response. This contract has it.
- **Signer rotation correctly invalidates stale sigs.** `test_SignerRotationInvalidatesOldSignerSigs` confirms intent and the implementation matches.
- **`_update` override list is correct.** Combines ERC1155 and ERC1155Supply correctly. ERC1155Burnable does not need to be in the override list because it does not override `_update` in OZ v5.
- **No upgradeability footgun.** Constructor-only initialization, no proxy, no `initializer` semantics, no UUPS authorization gap. This contract is a known shape forever.
- **Solidity 0.8.28 with overflow protection by default.** No `unchecked` blocks anywhere; all arithmetic is checked.

Recommendations summary

ID	Severity	Required for production?	Effort
M-01	Medium	Recommended before adding new booster tiers	1 hour (typehash change + test)
M-02	Medium	Recommended before adding new booster tiers	2 hours (storage + admin gate + test)

ID	Severity	Required for production?	Effort
L-01	Low	Defensive only	15 min
L-02	Low	Operational nicety	15 min
L-03	Low	Off-chain consumer ergonomics	5 min
L-04	Low	Repository hygiene	5 min (add LICENSE file)
I-01	Informational	Cleanup	1 min (delete file)
I-02	Informational	Style	1 min
I-03	Informational	Only if M-01 rejected	1 hour
I-04	Informational	Test coverage	2 hours

If the team wants a minimal-blast-radius PR landing the safe items, **L-04 (add LICENSE) + L-03 (event indexing) + I-01 (remove Modelfile) + I-02 (NatSpec)** are mergeable together with no semantic impact.

The medium items (M-01, M-02) deserve a separate design conversation before implementation, because they touch the EIP-712 typehash and add storage respectively.

Audit conducted by William Glynn, with primitive-assist from JARVIS. Methodology: TRP-Solidity primitive checklist (deposit-identity propagation, settlement-time binding, rate-of-change guards, collateral path independence, batch invariant verification, off-circulation registry, rebase-invariant accounting, post-upgrade init gate, settlement state durability, phantom array antipattern) plus integration primitives (identity divergence, dead guard antipattern, liveness coupling, enforced liveness signal, merkle commit-dispute-finalize). Each primitive was applied to the contract surface; findings reflect the full pass.